

Reviewer Pack — Specht Block Spread of Perm Exceeds Det Under Random Linear ...

Entry 48b7235cd4ff · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 09:38:10 UTC

Specht Block Spread of Perm Exceeds Det Under Random Linear Lift

Entry ID: 48b7235cd4ff **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 09:38:10 UTC

1. Conjecture

Field A (mathematical branch): Algebraic combinatorics: Specht-module projections via Murnaghan-Nakayama characters on the regular S_n -representation **Field B** (complexity object): $GL(n^2)$ -orbit spread of perm_n vs det_n in matching-coefficient basis (Mulmuley-Sohoni GCT, monotone setting)

Statement:

Let $X=(x_{ij})$ be an $n \times n$ matrix of formal variables. For a multilinear degree- n polynomial g in x_{ij} , define the matching coefficient vector $c(g) \in Q^{\wedge \{S_n\}}$ by $c_\sigma(g) = \prod_i x_{i, \sigma(i)}$. For each $\lambda \vdash n$, let P_λ be the projection onto the λ -isotypic component of $Q[S_n]$ using the central idempotent $e_\lambda = (f^{n!}/n!) \sum_\sigma \chi_\lambda(\sigma) \sigma$. Define the Specht block count $\rho(g) = \#\{\lambda \vdash n: \|P_\lambda(c(g))\|^2 / \|c(g)\|^2 > 10^{-9}\}$. Conjecture: $\rho(\text{perm}_n) = \rho(\text{det}_n) = 1$ for all $n \geq 2$; and for every $n \in \{4, 5, 6\}$, for a uniformly random invertible $L \in \{-1, 0, +1\}^{n^2 \times n^2}$, with probability $\geq 1 - 1/n$ we have $\rho(\text{perm}_n(L(X))) \geq \rho(\text{det}_n(L(X))) + 1$.

Rationale (proposer's reasoning):

Perm_n 's matching vector is the all-ones (max-entropy) direction in $Q^{\wedge \{S_n\}}$; det_n 's is the sign vector — both extremal but on opposite 1-dim irreps. A random linear lift $\bar{L}$ (the GCT analog of a LIFTING query-to-communication gadget) breaks the $S_n \times S_n$ grading that pins each to a single Specht block. Under random L , the trivial-rep-anchored perm should spread across STRICTLY MORE Specht blocks than the sign-rep-anchored det, because the sign vector's interaction with the regular representation under generic linear lift is more rigid (a Schur-Weyl asymmetry inherited from the Foata bijection inverting maj-inv).

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a55271770ecb55ef

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Sanity: $\rho(\text{perm}_n)=\rho(\text{det}_n)=1$ for $n \in \{4,5\}$. Main test: across 30 random invertible $L \in \{-1,0,+1\}^{n^2 \times n^2}$ per $n \in \{4,5\}$, count seeds where $\rho(\text{perm}_n \circ L) \geq \rho(\text{det}_n \circ L) + 1$. Support if $\geq 25/30$ for both n ; falsified if any single seed yields $\rho(\text{perm}_n \circ L) \leq \rho(\text{det}_n \circ L)$, or if sanity check fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Specht module isotypic decomposition permanent determinant matching coefficients symmetric group - Mulmuley Sohoni geometric complexity theory permanent determinant orbit Specht representation Murnaghan-Nakayama - random linear substitution permanent determinant symmetric group representation isotypic components GCT

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def perm_to_id(perm):
    return tuple(perm)

def id_to_perm(id):
    n = len(id)
    perm = [0] * n
    used = [False] * n
    for i in range(n):
        j = 0
        k = 0
        while True:
```

```

        if not used[j]:
            if k == id[i]:
                perm[i] = j
                used[j] = True
                break
            k += 1
        j += 1
    return tuple(perm)

def sign_of_permutation(perm):
    n = len(perm)
    inversions = 0
    for i in range(n):
        for j in range(i + 1, n):
            if perm[i] > perm[j]:
                inversions += 1
    return (-1) ** inversions

def matching_coefficient(matrix, g):
    n = len(matrix)
    coeff = [0] * factorial(n)
    for perm in itertools.permutations(range(n)):
        sign = sign_of_permutation(perm)
        value = g(perm)
        coeff[perm_to_id(perm)] += sign * value
    return coeff

def permanent(matrix):
    n = len(matrix)
    if n == 0:
        return 1
    if n == 1:
        return matrix[0][0]
    result = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        sign = (-1) ** j
        result += sign * matrix[0][j] * permanent(submatrix)
    return result

def projection_on_irrep(matrix, lambda_partition):
    n = len(matrix)
    f_lambda = Fraction(factorial(n), factorial(lambda_partition))
    coeff = matching_coefficient(matrix, lambda perm: 1)
    result = [0] * factorial(n)
    for sigma in itertools.permutations(range(n)):
        id_sigma = perm_to_id(sigma)
        sum_g_chi = 0
        for g in itertools.permutations(range(n)):
            id_g = perm_to_id(g)
            chi_lambda_g = 1 if lambda_partition == [1] * n else 0
            sum_g_chi += coeff[id_g] * chi_lambda_g
        result[id_sigma] = f_lambda ** 2 * abs(sum_g_chi) ** 2 / (f_lambda ** 2 * sum(coeff))
    return result

def run_trial(seed: int) -> dict:

```

```

random.seed(seed)

def identity_matrix(n):
    return [[1 if i == j else 0 for j in range(n)] for i in range(n)]

def perm_n(x):
    n = len(x)
    return sum([x[i][i] for i in range(n)])

def det_n(x):
    return permanent(x)

def random_invertible_matrix(n, values):
    while True:
        matrix = [[random.choice(values) for _ in range(n)] for _ in range(n)]
        if determinant(matrix) != 0:
            return matrix

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        sign = (-1) ** j
        det += sign * matrix[0][j] * determinant(submatrix)
    return det

n_values = [4, 5]
results = []

for n in n_values:
    rho_perm_n = sum(projection_on_irrep(identity_matrix(n), lambda_partition=[1]*n) > 1e-9 for _ in range(10))
    rho_det_n = sum(projection_on_irrep(identity_matrix(n), lambda_partition=[n]) > 1e-9 for _ in range(10))

    if rho_perm_n != 1 or rho_det_n != 1:
        return {
            "metric_name": "rho",
            "metric_value": None,
            "instances_tested": n_values.count(n),
            "conjecture_holds": False,
            "counterexample": f"Sanity check failed for n={n}: rho(perm_{n})={rho_perm_n}, rho(det_{n})={rho_det_n}"
        }

    results.append((n, rho_perm_n, rho_det_n))

for n in n_values:
    for _ in range(30):
        L = random_invertible_matrix(n * n, [-1, 0, 1])
        perm_coeff = matching_coefficient(identity_matrix(n), lambda perm: sum(L[i][j] * identity_matrix(n)[j][i] for j in range(n)))
        det_coeff = matching_coefficient(identity_matrix(n), lambda perm: sum(L[i][j] * identity_matrix(n)[j][i] for j in range(n)))

        rho_perm_n_L = sum(projection_on_irrep(perm_coeff, lambda_partition) > 1e-9 for lambda_partition in lambda_partitions(n))
        rho_det_n_L = sum(projection_on_irrep(det_coeff, lambda_partition) > 1e-9 for lambda_partition in lambda_partitions(n))

```

```

        if rho_perm_n_L <= rho_det_n_L:
            return {
                "metric_name": "rho",
                "metric_value": None,
                "instances_tested": n_values.count(n),
                "conjecture_holds": False,
                "counterexample": f"Counterexample found for n={n}: rho(perm_{n})(L)={rho_perm_n_L}"
            }

mean_rho = sum(rho_perm_n + rho_det_n for _, rho_perm_n, rho_det_n in results) / (len(results))
std_rho = math.sqrt(sum((rho_perm_n + rho_det_n - mean_rho) ** 2 for _, rho_perm_n, rho_det_n in results))

support_fraction = sum(rho_perm_n > rho_det_n for _, rho_perm_n, rho_det_n in results) / len(results)

return {
    "metric_name": "rho",
    "metric_value": mean_rho,
    "instances_tested": n_values.count(n),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": ""
}

def partitions(n):
    result = []
    def partition(n, k):
        if n == 0:
            result.append(k)
            return
        for i in range(1, n + 1):
            partition(n - i, k + [i])
    partition(n, [])
    return result

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rho = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results if r["metric_value"] is not None))

    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample='{<desc>}' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4e5c65c.py", line 189, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4e5c65c.py", line 126, in run_trial
    rho_perm_n = sum(projection_on_irrep(identity_matrix(n), lambda_partition=[1]*n) > 1e-
9 for lambda_partition in partitions(n)) / len(partitions(n))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4e5c65c.py", line 126, in <genexpr>
    rho_perm_n = sum(projection_on_irrep(identity_matrix(n), lambda_partition=[1]*n) > 1e-
9 for lambda_partition in partitions(n)) / len(partitions(n))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4e5c65c.py", line 79, in projection_on
    f_lambda = Fraction(factorial(n), factorial(lambda_partition))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d4e5c65c.py", line 22, in factorial
    for i in range(2, n + 1):
    ~~~~
TypeError: can only concatenate list (not "int") to list
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` before producing any data (factorial was called on a list instead of an integer), so neither the sanity check nor the main 30-seed trial was evaluated. | next: Fix the bug by computing the hook-length formula for f^λ from the partition (or pass $n=\text{sum}(\lambda)$) and rerun the sanity check for $n \in \{4,5\}$ before launching the random-lift trials.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	422145
2	preregistration	claude_max	opus	0	0	7165
3	novelty	claude_max	opus	0	0	4224
4	novelty	claude_max	opus	0	0	7548

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24840
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16446
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14856
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22330
9	judge	claude_max	opus	0	0	4369

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 523923 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in research/audit/48b7235cd4ff.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/48b7235cd4ff.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/48b7235cd4ff.tar.gz (if generated)